InterProcess Communication
IPC

Processes frequently need to communicate with other processes. For example, in a shell pipeline, the output of the first process must be passed to the second process, and so on down the line.

```
OS:>> ls | grep p1 | wc -l
2
```

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

InterProcess Communication IPC

Race conditions

Situations where two or more processes are reading or writing some shared data and the final result depends on who runs precisely when, are called <u>race conditions</u>. Debugging programs containing race conditions is no fun at all.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

InterProcess Communication
IPC

Too many cooks spoil the broth

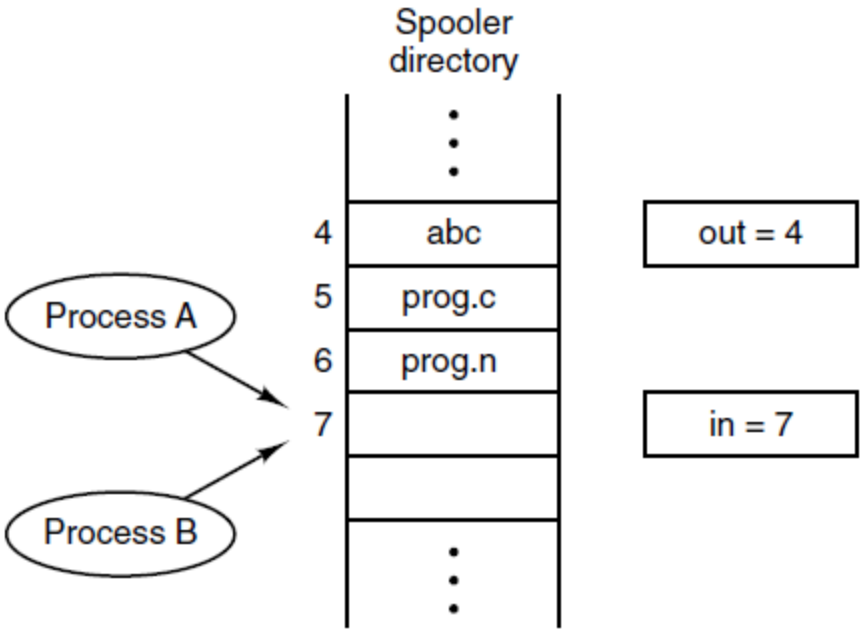
Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

InterProcess Communication IPC

Example : a print spooler

Slots 0 to 3 are empty (the files have already been printed) and slots 4 to 6 are full (with the names of files queued for printing). More or less simultaneously, processes *A* and *B* decide they want to queue a file for printing

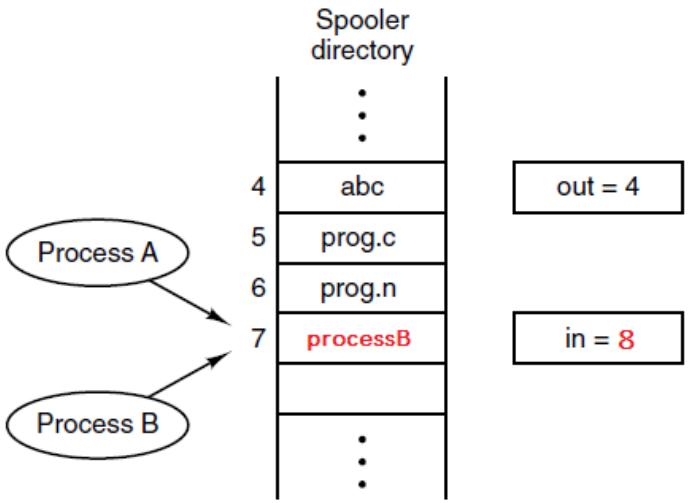
Shared variables, *out* which points to the next file to be printed, and *in*, which points to the next free slot in the directory.



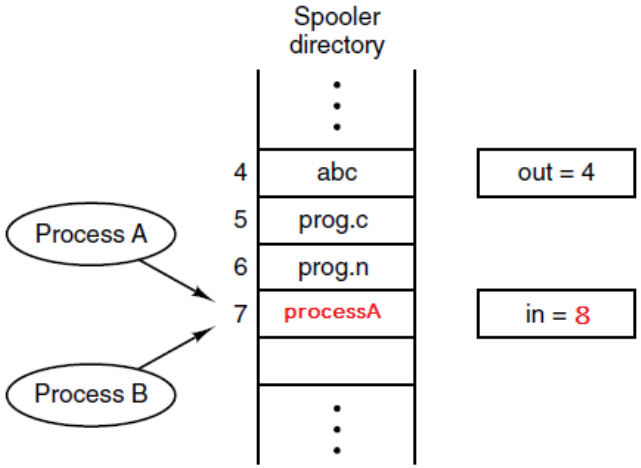
Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

InterProcess Communication IPC

```
Process A
Local variable next_free_slot = 7

A clock interrupt occurs.

Process B
Local variable next_free_slot = 7
It stores the name of its file in slot 7
```




```
A clock interrupt occurs.

Process A
Local variable next_free_slot = 7
It writes its file name in slot 7

Result:
Process B will never receive any output.
```



Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

IPC - Critical Regions

Mutual exclusion, some way of making sure that if one process is using a shared variable or file, the other processes will be excluded from doing the same thing.

That part of the program where the shared memory is accessed is called the critical region or critical section.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

IPC - Critical Regions

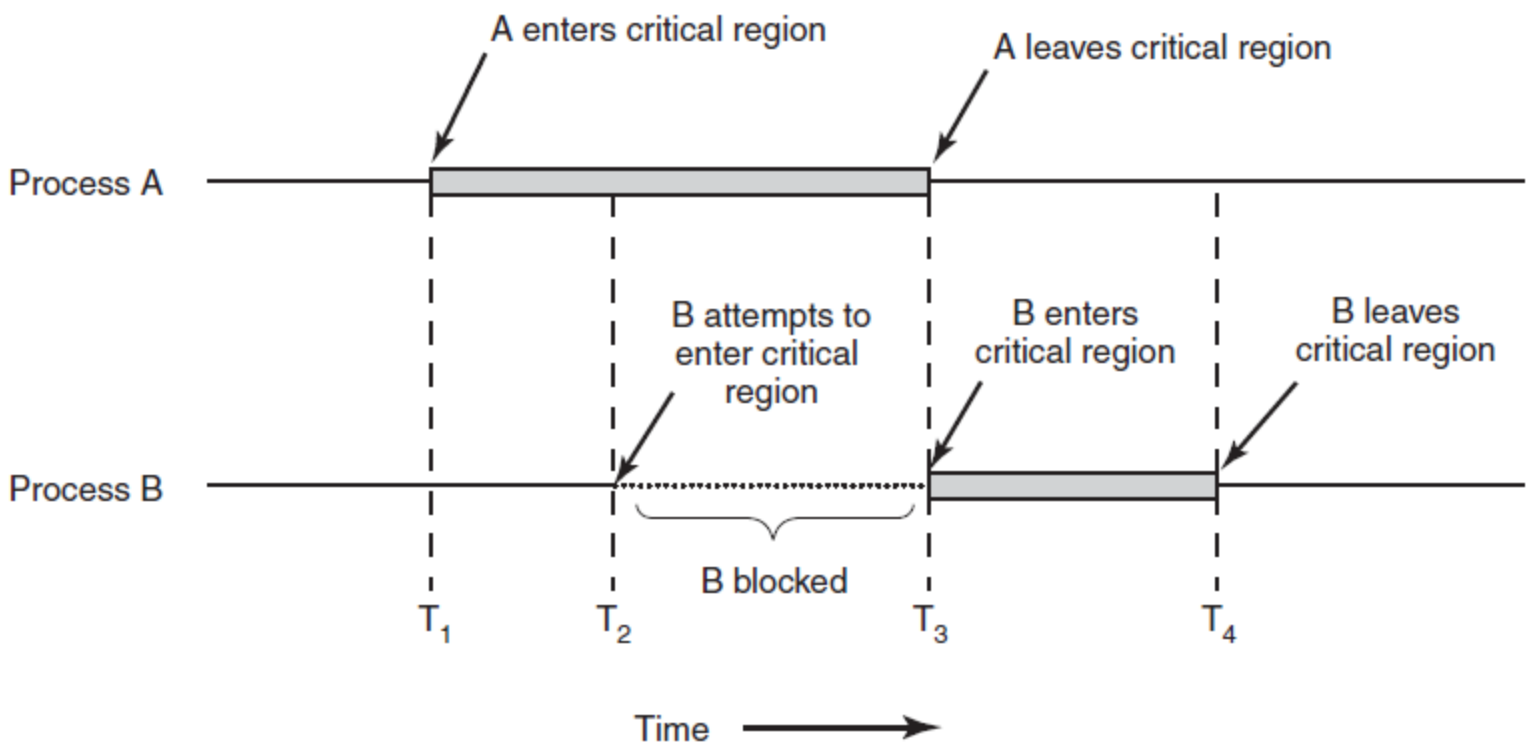
Here process *A* enters its critical region at time *T*1.
A little later, at time *T*2 process *B* attempts to enter its critical region but fails because another process is already in its critical region and we allow only one at a time.

Consequently, *B* is temporarily suspended until time *T*3 when *A* leaves its critical region, allowing *B* to enter immediately.

Eventually *B* leaves (at *T*4) and we are back to the original situation with no processes in their critical regions.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

IPC - Critical Regions



Mutual exclusion using critical regions.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

IPC - Critical Regions

Requirements to avoid race conditions:

1. No two processes may be simultaneously inside their critical regions.
2. No assumptions may be made about speeds or the number of CPUs.
3. No process running outside its critical region may block other processes.
4. No process should have to wait forever to enter its critical region.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

IPC - Critical Regions

No two processes may be simultaneously inside their critical regions

- This can be enforced using synchronization mechanisms such as locks, semaphores, or monitors.
- When a process enters the critical region, it must lock the resource, ensuring no other process can enter until it leaves and releases the lock.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

IPC - Critical Regions

No assumptions may be made about speeds or the number of CPUs

- **Independence from Hardware and Timing:** The correctness of the solution should not rely on specific hardware characteristics or timing assumptions.
- Processes may run at different speeds, and the number of CPUs may vary. The synchronization mechanism should work regardless of these factors.
- This ensures that the system remains robust and portable across different environments.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

IPC - Critical Regions

No process running outside its critical region may block other processes

- **Non-interference:** Processes not in their critical regions should not interfere with or delay other processes that need to enter their critical regions.
- This ensures that resources are not unnecessarily held, allowing other processes to proceed without unnecessary delays.
- For instance, a process that has completed its critical section should release any locks or signals, allowing other processes to access the shared resource.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

IPC - Critical Regions

No process should have to wait forever to enter its critical region

- **Avoidance of Starvation:** Every process should eventually get a chance to execute in its critical region, ensuring fairness.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

IPC -Mutual Exclusion with Busy Waiting

Continuously testing a variable until some value appears is called busy waiting.

It should usually be avoided, since it wastes CPU time. Only when there is a reasonable expectation that the wait will be short is busy waiting used.

A lock that uses busy waiting is called a spin lock.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

IPC -Mutual Exclusion with Busy Waiting

- Disabling Interrupts
- Lock Variables
- Strict Alternation
- Peterson’s Solution
- The TSL Instruction

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Mutual Exclusion with Busy Waiting: Disabling Interrupts

Disable all interrupts just after entering its critical region and re-enable them just before leaving it.

The CPU is only switched from process to process as a result of clock or other interrupts.

Once a process has disabled interrupts, it can examine and update the shared memory without fear that any other process will intervene.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Mutual Exclusion with Busy Waiting: Disabling Interrupts

Not good idea to give user processes the power to turn off interrupts. What if one of them did it, and never turned them on again? That could be the end of the system.

Multiprocessor (with two or more CPUs) disabling interrupts affects only the CPU that executed the disable instruction. The other ones will continue running and can access the shared memory.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Mutual Exclusion with Busy Waiting: Lock variables

Software solution:

Consider having a single, shared variable (lock), initially 0.

a) If Lock = 0 then the process set lock = 1 and enters critical region.

b) If Lock = 1 then the process just waits until it becomes 0.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Mutual Exclusion with Busy Waiting: Lock variables

Problem :

Process A
lock = 0, no chance of changing the lock value from 0 to 1.
A clock interrupt occurs
Process B
lock = 0 then lock = 1
A clock interrupt occurs
Process A
lock = 0 then lock = 1

Result:
Two processes will be in their critical regions at the same time.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Mutual Exclusion with Busy Waiting: Strict Alternation

```
while (TRUE) {
    while (turn != 0)        /* loop */ ;
    critical_region( );
    turn = 1;
    noncritical_region( );
}
```

(a)

```
while (TRUE) {
    while (turn != 1)        /* loop */ ;
    critical_region( );
    turn = 0;
    noncritical_region( );
}
```

(b)

A proposed solution to the critical region problem. (a) Process 0. (b) Process 1. In both cases, be sure to note the semicolons terminating the while statements.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Mutual Exclusion with Busy Waiting: Strict Alternation

Problem: $t_{P0_non\text{-}critical_region} > t_{P1_non\text{-}critical_region}$

Process 0 leaves its critical region and sets turn to 1, enters its non-critical region.

Process 1 enters its critical region, leaves its critical region and sets turn to 0.
Process 1 enters its non-critical region, quickly finishes its job and goes back to the while loop Since turn is 0, process 1 has to wait for process 0 to finish its non-critical region so that it can enter its critical region.

This violates the third condition of providing mutual exclusion:

No process running outside its critical region may block other processes

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Mutual Exclusion with Busy Waiting: Strict Alternation

```
OS:>> ./Strict_Alternation.exe
Process 1 , turn value =  0
Process 1 , turn value =  0
Process 1 - Critical_region
Process 1 - Noncritical_region
Process 2 - Critical_region
Process 1 , turn value =  1
Process 1 - Critical_region
Process 2 - Noncritical_region
Process 1 , turn value =  0
Process 1 - Noncritical_region
Process 2 - Critical_region
Process 1 , turn value =  1
Process 2 - Noncritical_region
Process 1 - Critical_region
Process 1 , turn value =  0
Process 1 - Noncritical_region
Process 2 - Critical_region
Process 1 , turn value =  1

OS:>>
```

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Mutual Exclusion with Busy Waiting: Peterson's Solution

Peterson's Solution defines the procedures for entering and leaving the critical region.

Before using the shared variables (i.e., before entering its critical region), each process calls *enter region* with its own process number, 0 or 1, as parameter. This call will cause it to wait, if need be, until it is safe to enter. After it has finished with the shared variables, the process calls *leave region* to indicate that it is done and to allow the other process to enter, if it so desires.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Mutual Exclusion with Busy Waiting: Peterson's Solution

```
#define FALSE  0
#define TRUE   1
#define N      2                        /* number of processes */

int turn;                               /* whose turn is it? */
int interested[N];                      /* all values initially 0 (FALSE) */

void enter_region(int process);         /* process is 0 or 1 */
{
      int other;                        /* number of the other process */

      other = 1 – process;              /* the opposite of process */
      interested[process] = TRUE;       /* show that you are interested */
      turn = process;                   /* set flag */
      while (turn == process && interested[other] == TRUE) /* null statement */ ;
}

void leave_region(int process)          /* process: who is leaving */
{
      interested[process] = FALSE;      /* indicate departure from critical region */
}
```

Peterson's solution for achieving mutual exclusion.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Mutual Exclusion with Busy Waiting: Peterson's Solution

Process 0

```
#define FALSE  0
#define TRUE   1
#define N      2

int turn;
int interested[N];
```

interested = [FALSE, FALSE]

```
void enter_region(int process);
{
        int other;

        other = 1 – process;
        interested[process] = TRUE
        turn = process;
        while (turn == process && interested[other] == TRUE);
}
```

process = 0

other = 1

interested[0] = TRUE

trun = 0

0 0 interested[1] = FALSE

```
void leave_region(int process)
{
        interested[process] = FALSE;
}
```

interested[0] = FALSE

Process 1

```
#define FALSE  0
#define TRUE   1
#define N      2

int turn;
int interested[N];
```

interested = [TRUE , FALSE]

```
void enter_region(int process);
{
        int other;

        other = 1 – process;
        interested[process] = TRUE
        turn = process;
        while (turn == process && interested[other] == TRUE);
}
```

process = 1

other = 0

interested[1] = TRUE

trun = 1

1 1 interested[0] = TRUE

```
void leave_region(int process)
{
        interested[process] = FALSE;
}
```

interested[1] = FALSE

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Mutual Exclusion with Busy Waiting: Peterson's Solution

```
> ./main
Process 0
Process 1
Process 0 - Critical_region
Process 0 - Noncritical_region
Process 1 - Critical_region
Process 0
Process 1 - Noncritical_region
Process 0 - Critical_region
Process 1
Process 0 - Noncritical_region
Process 1 - Critical_region
Process 0
Process 1 - Noncritical_region
Process 0 - Critical_region
Process 1
Process 0 - Noncritical_region
Process 1 - Critical_region
Process 0
Process 1 - Noncritical_region
Process 0 - Critical_region
Process 1
Process 0 - Noncritical_region
Process 1 - Critical_region
Process 0
Process 1 - Noncritical_region
Process 0 - Critical_region
Process 1
Process 0 - Noncritical_region
Process 1 - Critical_region
Process 0
Process 1 - Noncritical_region
```

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Mutual Exclusion with Busy Waiting: The TSL Instruction

<u>TSL : Test and Set Lock</u>

Provided by the hardware

lock = 0 free

lock = 1 busy

```
enter_region:
    TSL REGISTER, LOCK   REGISTER = lock then set LOCK = 1
    CMP REGISTER, #0
    JNE enter_region     If (REGISTER<>0) then Enter_region
    RET                  return
```

The operations of reading the word and storing into it are guaranteed to be indivisible no other processor can access the memory word until the instruction is finished.

<u>The CPU executing the TSL instruction locks the memory bus to prohibit other CPUs from accessing memory until it is done</u>.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Mutual Exclusion with Busy Waiting: The TSL Instruction

```
enter_region:
        TSL REGISTER,LOCK        | copy lock to register and set lock to 1
        CMP REGISTER,#0          | was lock zero?
        JNE enter_region         | if it was nonzero, lock was set, so loop
        RET                      | return to caller; critical region entered

leave_region:
        MOVE LOCK,#0             | store a 0 in lock
        RET                      | return to caller
```

TSL in a fictitious assembly language.

Entering and leaving a critical region using the TSL instruction.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Mutual Exclusion with Busy Waiting: The TSL Instruction

```
enter_region:
        MOVE REGISTER,#1            | put a 1 in the register
        XCHG REGISTER,LOCK          | swap the contents of the register and lock variable
        CMP REGISTER,#0             | was lock zero?
        JNE enter_region            | if it was non zero, lock was set, so loop
        RET                         | return to caller; critical region entered

leave_region:
        MOVE LOCK,#0                | store a 0 in lock
        RET                         | return to caller
```

In X86 CPU family, instruction prefix lock is used to ensure the atomicity of the instruction.

Entering and leaving a critical region using the XCHG instruction (Intel x86)

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Mutual Exclusion with Busy Waiting: The TSL Instruction

```c
#include <stdio.h>
#include <stdbool.h>
#include <stdatomic.h>

volatile bool lock = false;

void acquire_lock(volatile bool *lock) {
    // Spin until we acquire the lock
    while (__atomic_test_and_set(lock, __ATOMIC_ACQUIRE)); // busy wait
}

void release_lock(volatile bool *lock) {
    __atomic_clear(lock, __ATOMIC_RELEASE);
}

```

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Sleep and Wakeup

Both Peterson's solution and the solutions using TSL or XCHG are correct, but both have the defect of requiring busy waiting wasting CPU time.

No busy waiting, sleep and wakeup. Sleep is a system call that causes the caller to block, that is, be suspended until another process wakes it up.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Sleep and Wakeup
The Producer-Consumer Problem

Two processes share a common, fixed-size buffer. One of them, the producer, puts information into the buffer, and the other one, the consumer, takes it out.

If the buffer is full then the producer goes to sleep until the consumer removes one or more items and wakes it up.

If the buffer is empty then the consumer goes to sleep until the producer puts something in the buffer and wakes it up.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Sleep and Wakeup
The Producer-Consumer Problem

```
#define N 100                                   /* number of slots in the buffer */
int count = 0;                                  /* number of items in the buffer */

void producer(void)
{
        int item;

        while (TRUE) {                          /* repeat forever */
                item = produce_item( );         /* generate next item */
                if (count == N) sleep( );       /* if buffer is full, go to sleep */
                insert_item(item);              /* put item in buffer */
                count = count + 1;              /* increment count of items in buffer */
                if (count == 1) wakeup(consumer); /* was buffer empty? */
        }
}


void consumer(void)
{
```

The producer-consumer problem
with a fatal race condition.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Sleep and Wakeup
The Producer-Consumer Problem

```
        if (count == 1) wakeup(consumer);    /* was buffer empty? */
        }
}


void consumer(void)
{
        int item;

        while (TRUE) {                          /* repeat forever */
                if (count == 0) sleep( );       /* if buffer is empty, got to sleep */
                item = remove_item( );          /* take item out of buffer */
                count = count – 1;              /* decrement count of items in buffer */
                if (count == N – 1) wakeup(producer);  /* was buffer full? */
                consume_item(item);             /* print item */
        }
}
```

The producer-consumer problem
with a fatal race condition.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Sleep and Wakeup
The Producer-Consumer Problem

Race condition

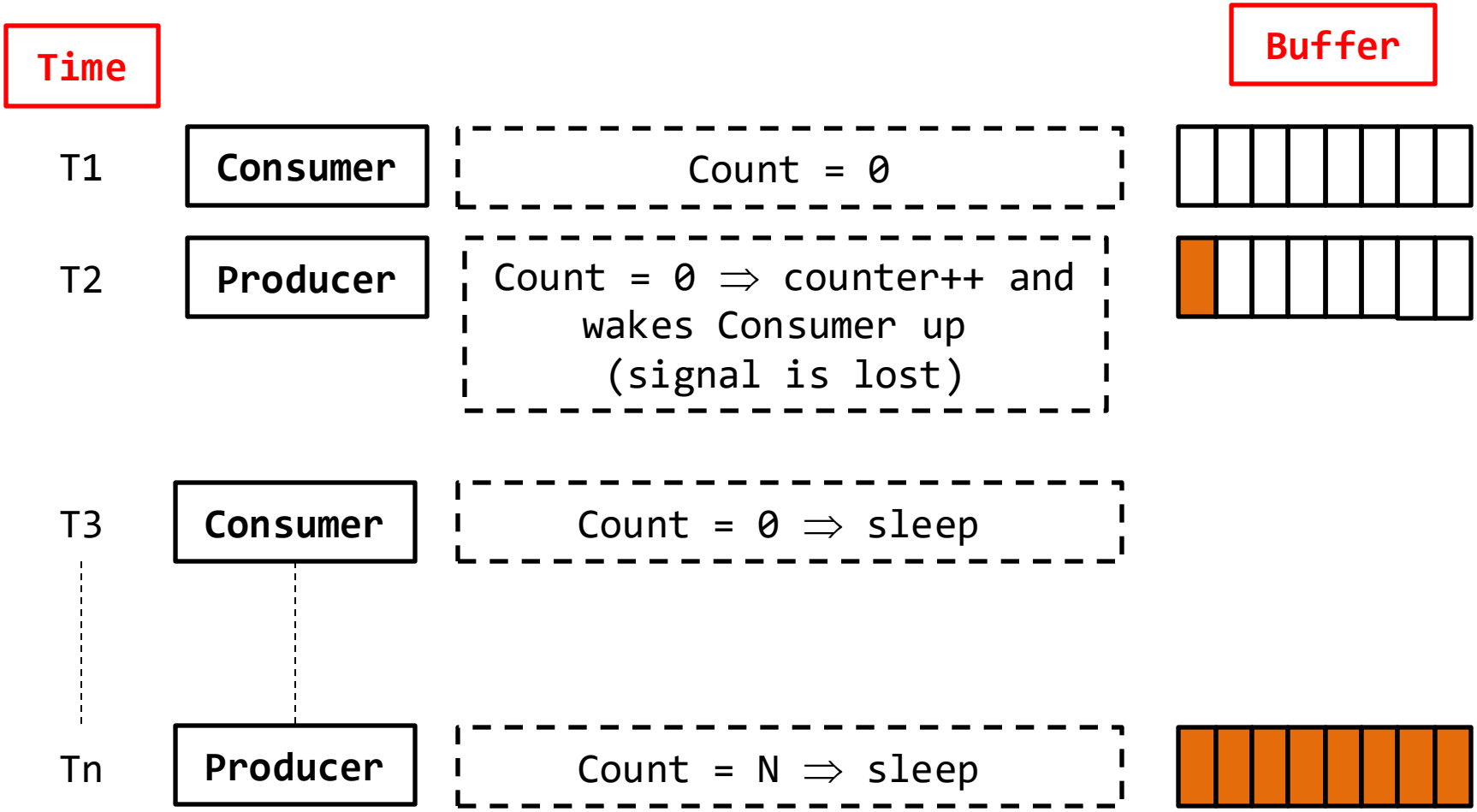
count : the number of items in the buffer
N : the maximum capacity of the buffer

The buffer is empty and the consumer has just read count to see if it is 0. At that instant, the scheduler decides to stop running the consumer temporarily and start running the producer.

The producer inserts an item in the buffer, increments count, and notices that it is now 1. Reasoning that count was just 0, and thus the consumer must be sleeping, the producer calls wakeup to wake the consumer up. Unfortunately, the consumer is not yet logically asleep, so the wakeup signal is lost. When the consumer next runs, it will test the value of count it previously read, find it to be 0, and go to sleep. Sooner or later the producer will fill up the buffer and also go to sleep. Both will sleep forever.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Sleep and Wakeup
The Producer-Consumer Problem

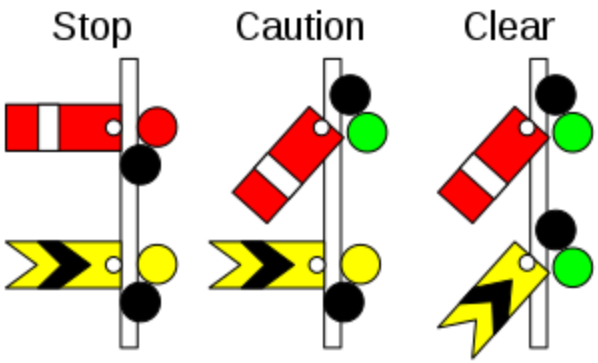


Semaphores

The idea comes from train semaphores to signal whether track is free. Introduced by Dijkstra in 1965. A new technique for managing concurrent processes by employing the value of an integer variable called semaphore and two atomic operations P and V (down and up) to synchronize the progress of interacting processes.

In an atomic operation it is guaranteed that once a semaphore operation has started, no other process can access the semaphore until the operation has completed or blocked.

```
P(s) :
 If (s == 0)
    sleep
 else
    s = s - 1
V(s):
    s = s + 1
```



Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Semaphores

Types of Semaphores

- Semaphores are mainly of two types:
  Binary Semaphore used for mutual exclusion: it takes two values of 0 or 1 and is often called Mutex.

- Counting Semaphores: These are used to implement bounded concurrency.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Semaphores

```
#define N 100                          /* number of slots in the buffer */
typedef int semaphore;                 /* semaphores are a special kind of int */
semaphore mutex = 1;                   /* controls access to critical region */
semaphore empty = N;                   /* counts empty buffer slots */
semaphore full = 0;                    /* counts full buffer slots */

void producer(void)
{
    int item;

    while (TRUE) {                     /* TRUE is the constant 1 */
        item = produce_item( );        /* generate something to put in buffer */
        down(&empty);                  /* decrement empty count */
        down(&mutex);                  /* enter critical region */
        insert_item(item);             /* put new item in buffer */
        up(&mutex);                    /* leave critical region */
        up(&full);                     /* increment count of full slots */
    }
}

void consumer(void)
```

The producer-consumer problem using semaphores.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Semaphores

```
            up(&full);                          /* increment count of full slots */
        }
}


void consumer(void)
{
        int item;

        while (TRUE) {                          /* infinite loop */
            down(&full);                        /* decrement full count */
            down(&mutex);                       /* enter critical region */
            item = remove_item();               /* take item from buffer */
            up(&mutex);                         /* leave critical region */
            up(&empty);                         /* increment count of empty slots */
            consume_item(item);                 /* do something with the item */
        }
}
```

The producer-consumer problem using semaphores.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Mutexes

When the semaphore's ability to count is not needed, a simplified version of the semaphore, called a mutex. A mutex is a shared variable that can be in one of two states: unlocked or locked.

Because mutexes are so simple, they can easily be implemented in user space provided that a TSL or XCHG instruction is available.

Consequently, only 1 bit is required to represent it, but in practice an integer often is used, with 0 meaning unlocked and all other values meaning locked.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Mutexes

```
mutex_lock:
        TSL REGISTER,MUTEX      | copy mutex to register and set mutex to 1
        CMP REGISTER,#0         | was mutex zero?
        JZE ok                  | if it was zero, mutex was unlocked, so return
        CALL thread_yield       | mutex is busy; schedule another thread
        JMP mutex_lock          | try again
ok:     RET                     | return to caller; critical region entered

mutex_unlock:
        MOVE MUTEX,#0           | store a 0 in mutex
        RET                     | return to caller
```

Implementation of *mutex_lock* and *mutex_unlock*.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Mutexes

```
mutex_lock:
        TSL REGISTER,MUTEX      | copy mutex to register and set mutex to 1
        CMP REGISTER,#0         | was mutex zero?
        JZE ok                  | if it was zero, mutex was unlocked, so return
        CALL thread_yield       | mutex is busy; schedule another thread
        JMP mutex_lock          | try again
ok:     RET                     | return to caller; critical region entered

mutex_unlock:
        MOVE MUTEX,#0           | store a 0 in mutex
        RET                     | return to caller
```

```
DESCRIPTION
       pthread_yield()  causes  the calling thread to relinquish
       the CPU.  The thread is placed at  the  end  of  the  run
       queue  for  its  static  priority  and  another thread is
       scheduled   to   run.    For   further    details,    see
       sched_yield(2)
```

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Mutexes in Pthreads

Thread call	Description
Pthread_mutex_init	Create a mutex
Pthread_mutex_destroy	Destroy an existing mutex
Pthread_mutex_lock	Acquire a lock or block
Pthread_mutex_trylock	Acquire a lock or fail
Pthread_mutex_unlock	Release a lock

Some of the Pthreads calls relating to mutexes.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Mutexes

In addition to mutexes, Pthreads offers a second synchronization mechanism: condition variables. Mutexes are good for allowing or blocking access to a critical region. Condition variables allow threads to block due to some condition not being met. Almost always the two methods are used together.

It is also worth noting that condition variables (unlike semaphores) have no memory. If a signal is sent to a condition variable on which no thread is waiting, the signal is lost.

Two primary operations of a condition variable: wait and signal. A thread that wants to wait on a condition variable must first acquire a mutex that protects access to the shared resource.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Mutexes in Pthreads

pthread_cond_wait method releases the mutex and puts the thread to sleep until another thread signals the condition variable.

pthread_cond_signal method is used to signal the condition variable, waking up at least one waiting thread.

Thread call	Description
Pthread_cond_init	Create a condition variable
Pthread_cond_destroy	Destroy a condition variable
Pthread_cond_wait	Block waiting for a signal
Pthread_cond_signal	Signal another thread and wake it up
Pthread_cond_broadcast	Signal multiple threads and wake all of them

Some of the Pthreads calls relating to condition variables.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Mutexes in Pthreads

```
#include <stdio.h>
#include <pthread.h>
#define MAX 1000000000                          /* how many numbers to produce */
pthread_mutex_t the_mutex;
pthread_cond_t condc, condp;                    /* used for signaling */
int buffer = 0;                                 /* buffer used between producer and consumer */

void *producer(void *ptr)                       /* produce data */
{       int i;

        for (i= 1; i <= MAX; i++) {
                pthread_mutex_lock(&the_mutex);  /* get exclusive access to buffer */
                while (buffer != 0) pthread_cond_wait(&condp, &the_mutex);
                buffer = i;                      /* put item in buffer */
                pthread_cond_signal(&condc);     /* wake up consumer */
                pthread_mutex_unlock(&the_mutex); /* release access to buffer */
        }
        pthread_exit(0);
}
```

Using threads to solve the producer-consumer problem.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Mutexes in Pthreads

```
        pthread_exit(0);
}

void *consumer(void *ptr)                          /* consume data */
{       int i;

        for (i = 1; i <= MAX; i++) {
                pthread_mutex_lock(&the_mutex);    /* get exclusive access to buffer */
                while (buffer ==0 ) pthread_cond_wait(&condc, &the_mutex);
                buffer = 0;                        /* take item out of buffer */
                pthread_cond_signal(&condp);       /* wake up producer */
                pthread_mutex_unlock(&the_mutex); /* release access to buffer */
        }
        pthread_exit(0);
}

int main(int argc, char **argv)
```

Using threads to solve the producer-consumer problem.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Mutexes in Pthreads

```
        pthread_exit(0);
}

int main(int argc, char **argv)
{
        pthread_t pro, con;
        pthread_mutex_init(&the_mutex, 0);
        pthread_cond_init(&condc, 0);
        pthread_cond_init(&condp, 0);
        pthread_create(&con, 0, consumer, 0);
        pthread_create(&pro, 0, producer, 0);
        pthread_join(pro, 0);
        pthread_join(con, 0);
        pthread_cond_destroy(&condc);
        pthread_cond_destroy(&condp);
        pthread_mutex_destroy(&the_mutex);
}
```

Using threads to solve the producer-consumer problem.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Mutexes in Pthreads

```
Producer is waiting, buffer = 16
consumed : 16
Consumer is waiting, buffer = 0
Produced : 4
Producer is waiting, buffer = 4
consumed : 4
Consumer is waiting, buffer = 0
Produced : 2
Producer is waiting, buffer = 2
consumed : 2
Consumer is waiting, buffer = 0
Produced : 0
Produced : 12
Producer is waiting, buffer = 12
consumed : 12
Consumer is waiting, buffer = 0
Produced : 9
Producer is waiting, buffer = 9
consumed : 9
Consumer is waiting, buffer = 0
Produced : 15
Producer is waiting, buffer = 15
consumed : 15
Consumer is waiting, buffer = 0
Produced : 7
Producer is waiting, buffer = 7
consumed : 7
Consumer is waiting, buffer = 0
Produced : 14
Producer is waiting, buffer = 14
consumed : 14
Consumer is waiting, buffer = 0
Produced : 6
Producer is waiting, buffer = 6
consumed : 6
Consumer is waiting, buffer = 0
```

Monitors
Java Threads

1. Extending the Thread Class
2. Implementing Runnable Interface
3. Using Lambda Expressions

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Monitors

```java
class MyThread extends Thread {
    static void waitThread(int msec){
        try{
            Thread.sleep(msec);
        }catch(InterruptedException e){
            e.printStackTrace();
        }
    }
    public void run(){
        String threadName = Thread.currentThread().getName();
        System.out.println(threadName);
            if (threadName == "T1")
                Main.func(5000, threadName);
            else if (threadName == "T2")
                Main.func(1000, threadName);
            else
                Main.func(100, threadName);
    }
}
public class Main{
    static void func(int msec, String tName){
        System.out.println(tName + " started");
        MyThread.waitThread(msec);
        System.out.println(tName + " terminted" );
    }
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        t1.setName( "T1");
        t1.start();

        MyThread t2 = new MyThread();
        t2.setName( "T2");
        t2.start();

        MyThread t3 = new MyThread();
        t3.setName( "T3");
        t3.start();
    }
}
```

```
T1
T3
T2
T2 started
T1 started
T3 started
T3 terminted
T2 terminted
T1 terminted
```

Non-synchronized method

Monitors

```java
class MyThread extends Thread {
    static void waitThread(int msec){
        try{
            Thread.sleep(msec);
        }catch(InterruptedException e){
            e.printStackTrace();
        }
    }
    public void run(){
        String threadName = Thread.currentThread().getName();
        System.out.println(threadName);
            if (threadName == "T1")
                Main.func(5000, threadName);
            else if (threadName == "T2")
                Main.func(1000, threadName);
            else
                Main.func(100, threadName);
    }
}
public class Main{
    static synchronized void func(int msec, String tName){
        System.out.println(tName + " started");
        MyThread.waitThread(msec);
        System.out.println(tName + " terminted" );
    }
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        t1.setName( "T1");
        t1.start();

        MyThread t2 = new MyThread();
        t2.setName( "T2");
        t2.start();

        MyThread t3 = new MyThread();
        t3.setName( "T3");
        t3.start();
    }
}
```

Synchronized method

```
T1
T3
T2
T1 started
T1 terminted
T2 started
T2 terminted
T3 started
T3 terminted
```

Monitors

A monitor is a collection of procedures, variables, and data structures that are all grouped together in a special kind of module or package. Processes may call the procedures in a monitor whenever they want to, but they cannot directly access the monitor's internal data structures from procedures declared outside the monitor.

Monitors have an important property that makes them useful for achieving mutual exclusion: only one process can be active in a monitor at any instant.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Monitors

The automatic mutual exclusion on monitor procedures guarantees that if, say, the producer inside a monitor procedure discovers that the buffer is full, it will be able to complete the wait operation without having to worry about the possibility that the scheduler may switch to the consumer just before the wait completes.

The consumer will not even be let into the monitor at all until the wait is finished and the producer has been marked as no longer runnable.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Monitors

```
public class ProducerConsumer {
        static final int N = 100;       // constant giving the buffer size
        static producer p = new producer( );    // instantiate a new producer thread
        static consumer c = new consumer( ); // instantiate a new consumer thread
        static our_monitor mon = new our_monitor( );       // instantiate a new monitor

        public static void main(String args[ ]) {
           p.start( );      // start the producer thread
           c.start( );      // start the consumer thread
        }

        static class producer extends Thread {
           public void run( )  { // run method contains the thread code
              int item;
              while (true) {      // producer loop
                 item = produce_item( );
                 mon.insert(item);
              }
           }
           private int produce_item( ) { ... }      // actually produce
        }

        static class consumer extends Thread {
```

A solution to the producer-consumer problem in Java.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Monitors

```
    private int produce_item( ) { ... }      // actually produce
}

static class consumer extends Thread {
    public void run( )  { run method contains the thread code
        int item;
        while (true) {      // consumer loop
            item = mon.remove( );
            consume_item (item);
        }
    }
    private void consume_item(int item) { ... }// actually consume
}

static class our_monitor {   // this is a monitor
    private int buffer[ ] = new int[N];
    private int count = 0, lo = 0, hi = 0;   // counters and indices

    public synchronized void insert(int val) {
```

A solution to the producer-consumer problem in Java.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

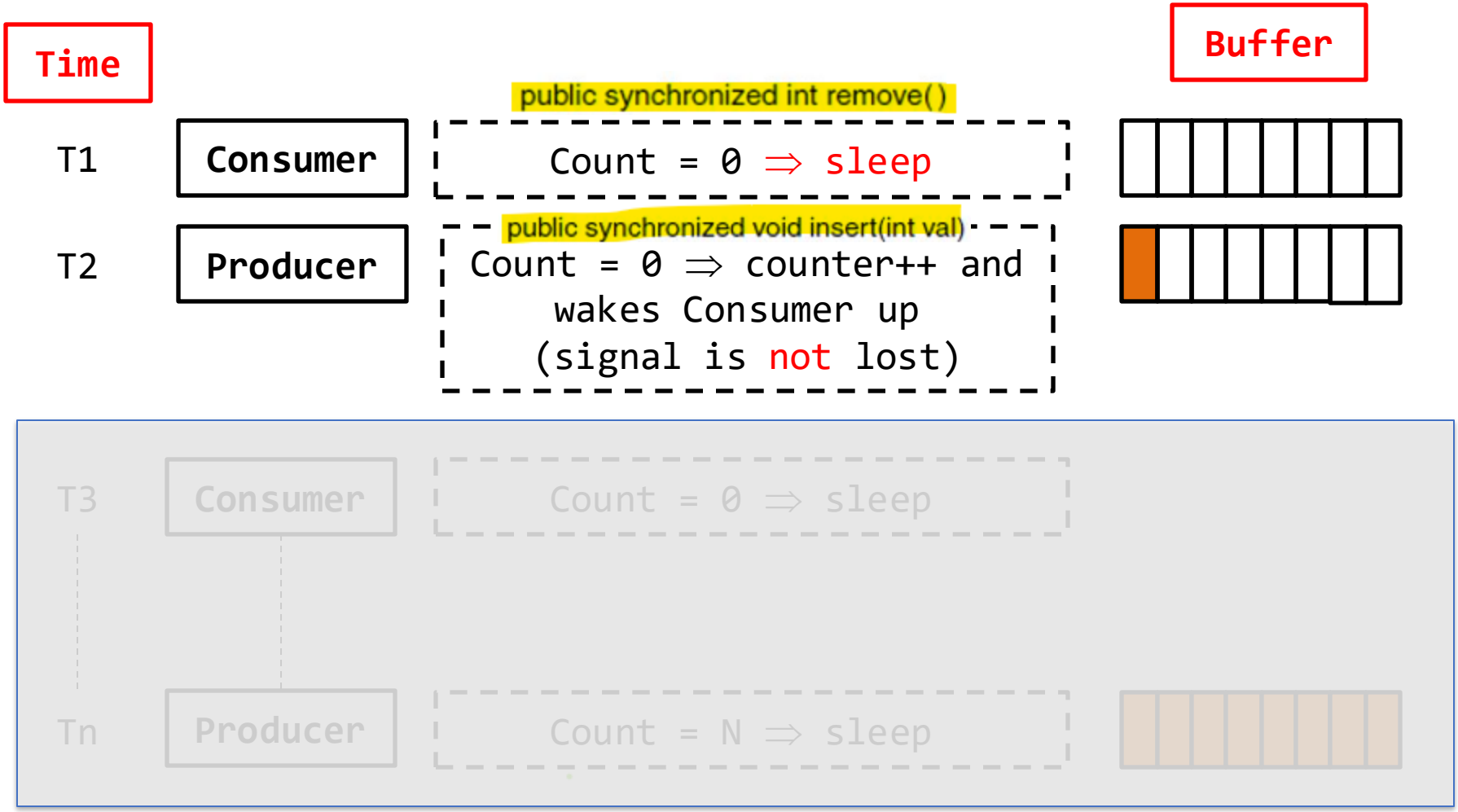
Monitors

```
        if (count == N) go_to_sleep( );     // if the buffer is full, go to sleep
        buffer [hi] = val; // insert an item into the buffer
        hi = (hi + 1) % N;        // slot to place next item in
        count = count + 1;        // one more item in the buffer now
        if (count == 1) notify( );        // if consumer was sleeping, wake it up
    }

    public synchronized int remove( ) {
        int val;
        if (count == 0) go_to_sleep( );     // if the buffer is empty, go to sleep
        val = buffer [lo]; // fetch an item from the buffer
        lo = (lo + 1) % N;        // slot to fetch next item from
        count = count – 1;        // one few items in the buffer
        if (count == N – 1) notify( ); // if producer was sleeping, wake it up
        return val;
    }
    private void go_to_sleep( ) { try{wait( );} catch(InterruptedException exc) {};}
}
```

A solution to the producer-consumer problem in Java.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Sleep and Wakeup
The Producer-Consumer Problem



Message Passing

- Semaphores are too low level and monitors are not usable except in a few programming languages.

- None of the primitives allow information exchange between machines.

Message passing is commonly used in parallel programming systems. One well-known message-passing system, e.g., is MPI (Message-Passing Interface), widely used for scientific computing.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

The Producer-Consumer Problem with Message Passing

- A total of N messages is used, analogous to the N slots in a shared-memory buffer.
- The consumer starts out by sending N empty messages to the producer.
- Whenever the producer has an item to give to the consumer, it takes an empty message and sends back a full one.

If the producer works faster than the consumer, all the messages will end up full, waiting for the consumer; the producer will be blocked, waiting for an empty to come back.

If the consumer works faster, then the reverse happens: all the messages will be empties waiting for the producer to fill them up; the consumer will be blocked, waiting for a full message.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

The Producer-Consumer Problem with Message Passing

```
#define N 100                                   /* number of slots in the buffer */

void producer(void)
{
        int item;
        message m;                              /* message buffer */

        while (TRUE) {
                item = produce_item( );         /* generate something to put in buffer */
                receive(consumer, &m);          /* wait for an empty to arrive */
                build_message(&m, item);        /* construct a message to send */
                send(consumer, &m);             /* send item to consumer */
        }
}

void consumer(void)
```

The producer-consumer problem with *N* messages.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

The Producer-Consumer Problem with Message Passing

```
            send(consumer, &m);                /* send item to consumer */
        }
    }

    void consumer(void)
    {
        int item, i;
        message m;

        for (i = 0; i < N; i++) send(producer, &m);  /* send N empties */
        while (TRUE) {
            receive(producer, &m);             /* get message containing item */
            item = extract_item(&m);           /* extract item from message */
            send(producer, &m);                /* send back empty reply */
            consume_item(item);                /* do something with the item */
        }
    }
```

The producer-consumer problem with *N* messages.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

MPI

```
~$ sudo apt install mpich
```

```
~$ mpicc hello_mpi_1.c  -o Hello1
```

```
~$ lscpu | grep Socket
Socket(s):                       1
~$ lscpu | grep Core
Core(s) per socket:              2
~$ lscpu | grep Thread
Thread(s) per core:              2
~$
```

```c
#include <mpi.h>
#include <stdio.h>

int main(int argc, char **argv){
  int number_of_procs;
  int my_id;

  MPI_Init( &argc, & argv );
  MPI_Comm_size(MPI_COMM_WORLD, &number_of_procs);
  MPI_Comm_rank(MPI_COMM_WORLD, &my_id);

  if(my_id==0)
    printf("Number of processors: %d \n", number_of_procs);

  printf("Hello World! processor ID: %d \n", my_id);
  MPI_Finalize();
  return 0;
}
```

MPI

```
~$ mpirun  -np 2      ./Hello1
Hello World! processor ID: 1
Number of processors: 2
Hello World! processor ID: 0
~$
```

```
~$ mpirun  --use-hwthread-cpus      ./Hello1
Number of processors: 4
Hello World! processor ID: 0
Hello World! processor ID: 1
Hello World! processor ID: 3
Hello World! processor ID: 2
~$
```

```
-use-hwthread-cpus, --use-hwthread-cpus
       Use hardware threads as independent cpus.
```

MPI

```c
#include <stdio.h>
#include <mpi.h>

int main(int argc, char** argv) {
    int rank, size;
    int message;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // The message to be broadcasted
    if (rank == 0) {
        message = 42;
        printf("Process 0 broadcasting message: %d\n", message);
    } else {
        message = 0; // Received message placeholder for non-root processes
    }

    // Broadcasting the message from process 0 to all other processes
    MPI_Bcast(&message, 1, MPI_INT, 0, MPI_COMM_WORLD);

    // All processes print the received message
    printf("Process %d received message: %d\n", rank, message);

    MPI_Finalize();
    return 0;
}
```

```
asp@DESKTOP-U18BSDT:~/class_codes/c_codes/c_examples$ mpirun -np 4  ./mpibc
Process 0 broadcasting message: 42
Process 0 received message: 42
Process 1 received message: 42
Process 2 received message: 42
Process 3 received message: 42
asp@DESKTOP-U18BSDT:~/class_codes/c_codes/c_examples$
```

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Barriers

Synchronization mechanism for groups of processes used in applications with many phases.

Have the rule that no process may proceed into the next phase until all processes are ready to proceed to the next phase.

Example:

Relaxation problem in physics or engineering

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

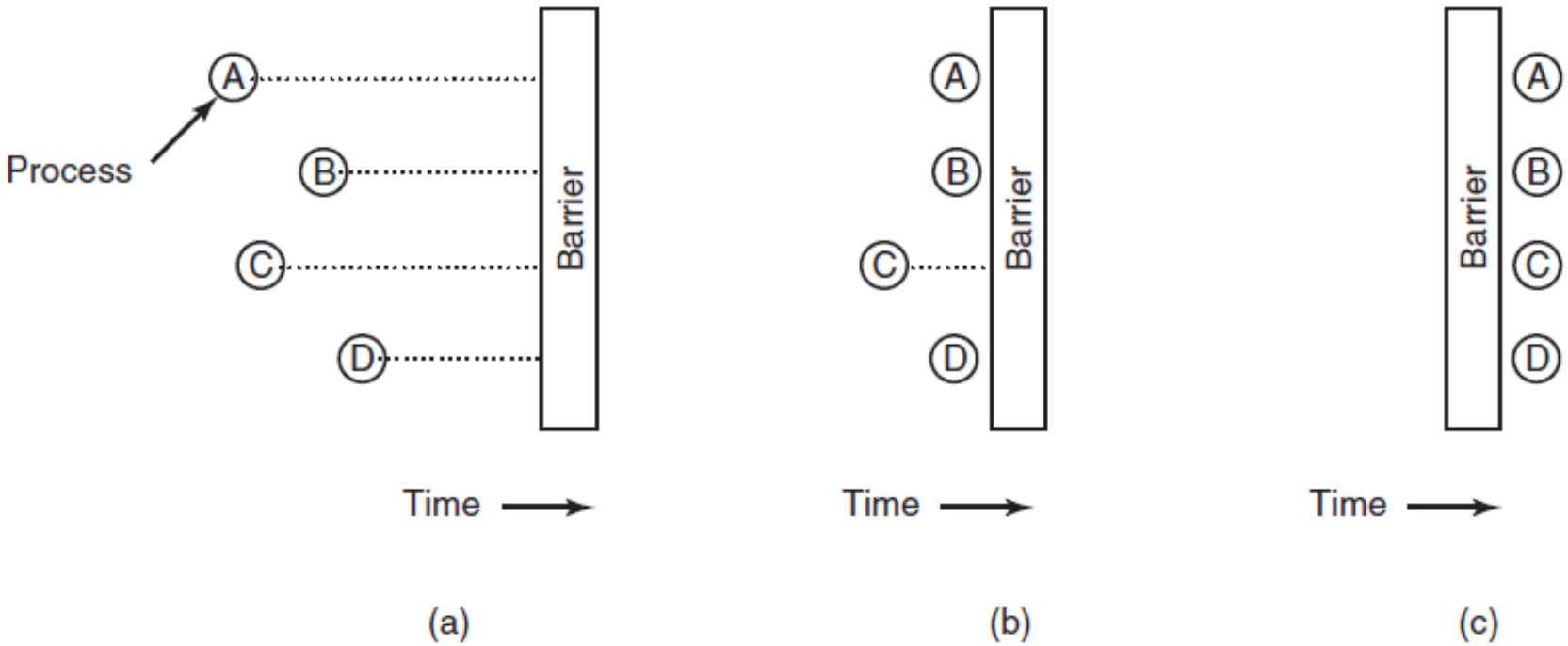
Barriers

Use of a barrier

(a) Processes approaching a barrier.

(b) All processes but one blocked at the barrier.

(c) When the last process arrives at the barrier, all of them are let through.



Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Barriers

```
int pthread_barrier_init(
                    pthread_barrier_t * barrier,
                    const pthread_barrierattr_t * attr
                    unsigned int count )

int pthread_barrier_wait( pthread_barrier_t * barrier )

int pthread_barrier_destroy(pthread_barrier_t *barrier)
```

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

The Readers and Writers Problem

An airline reservation system, with many competing processes wishing to read and write it.

It is acceptable to have multiple processes reading the database at the same time, but if one process is updating (writing) the database, no other processes may have access to the database, not even readers.

The question is how do you program the readers and the writers?

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

The Readers and Writers Problem

```
typedef int semaphore;                  /* use your imagination */
semaphore mutex = 1;                    /* controls access to 'rc' */
semaphore db = 1;                       /* controls access to the database */
int rc = 0;                             /* # of processes reading or wanting to */

void reader(void)
{
        while (TRUE) {                  /* repeat forever */
                down(&mutex);           /* get exclusive access to 'rc' */
                rc = rc + 1;            /* one reader more now */
                if (rc == 1) down(&db); /* if this is the first reader ... */
                up(&mutex);             /* release exclusive access to 'rc' */
                read_data_base( );      /* access the data */
                down(&mutex);           /* get exclusive access to 'rc' */
                rc = rc – 1;            /* one reader fewer now */
                if (rc == 0) up(&db);   /* if this is the last reader ... */
                up(&mutex);             /* release exclusive access to 'rc' */
                use_data_read( );       /* noncritical region */
        }
}

void writer(void)
```

A solution to the readers and writers problem.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

The Readers and Writers Problem

```
            use_data_read();            /* noncritical region */
        }
}


void writer(void)
{
        while (TRUE) {                  /* repeat forever */
            think_up_data();            /* noncritical region */
            down(&db);                  /* get exclusive access */
            write_data_base();          /* update the data */
            up(&db);                    /* release exclusive access */
        }
}
```

A solution to the readers and writers problem.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

The Readers and Writers Problem

```
typedef int semaphore;
semaphore mutex = 1;
semaphore db = 1;
int rc = 0;
```

Reader 1

```
void reader(void)
{
    while (TRUE) {
        down(&mutex);
        rc = rc + 1;
        if (rc == 1) down(&db);
        up(&mutex);
        read_data_base( );
        down(&mutex);
        rc = rc – 1;
        if (rc == 0) up(&db);
        up(&mutex);
        use_data_read( );
    }
}
```

Reader 2

```
void reader(void)
{
    while (TRUE) {
        down(&mutex);
        rc = rc + 1;
        if (rc == 1) down(&db);
        up(&mutex);
        read_data_base( );
        down(&mutex);
        rc = rc – 1;
        if (rc == 0) up(&db);
        up(&mutex);
        use_data_read( );
    }
}
```

Reader 3

```
void reader(void)
{
    while (TRUE) {
        down(&mutex);
        rc = rc + 1;
        if (rc == 1) down(&db);
        up(&mutex);
        read_data_base( );
        down(&mutex);
        rc = rc – 1;
        if (rc == 0) up(&db);
        up(&mutex);
        use_data_read( );
    }
}
```

Writer

```
void writer(void)
{
    while (TRUE) {
        think_up_data( );
        down(&db);
        write_data_base( );
        up(&db);
    }
}
```

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

The Readers and Writers Problem

```
typedef int semaphore;
semaphore mutex = 1;
semaphore db = 1;
int rc = 0;
```

Reader 1

```
void reader(void)
{
    while (TRUE) {
        down(&mutex);
        rc = rc + 1;
        if (rc == 1) down(&db);
        up(&mutex);
        read_data_base();
        down(&mutex);
        rc = rc – 1;
        if (rc == 0) up(&db);
        up(&mutex);
        use_data_read();
    }
}
```

Reader 2

```
void reader(void)
{
    while (TRUE) {
        down(&mutex);
        rc = rc + 1;
        if (rc == 1) down(&db);
        up(&mutex);
        read_data_base();
        down(&mutex);
        rc = rc – 1;
        if (rc == 0) up(&db);
        up(&mutex);
        use_data_read();
    }
}
```

Reader 3

```
void reader(void)
{
    while (TRUE) {
        down(&mutex);
        rc = rc + 1;
        if (rc == 1) down(&db);
        up(&mutex);
        read_data_base();
        down(&mutex);
        rc = rc – 1;
        if (rc == 0) up(&db);
        up(&mutex);
        use_data_read();
    }
}
```

```
mutex = 0
rc = 1
db = 0
```

mutex = 0 Reader 1 or Reader 2 -> sleep

Writer

```
void writer(void)
{
    while (TRUE) {
        think_up_data();
        down(&db);
        write_data_base();
        up(&db);
    }
}
```

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

The Readers and Writers Problem

```
typedef int semaphore;
semaphore mutex = 1;
semaphore db = 1;
int rc = 0;
```

Reader 1

```
void reader(void)
{
    while (TRUE) {
        down(&mutex);
        rc = rc + 1;
        if (rc == 1) down(&db);
        up(&mutex);
        read_data_base();
        down(&mutex);
        rc = rc – 1;
        if (rc == 0) up(&db);
        up(&mutex);
        use_data_read();
    }
}
```

Reader 2

```
void reader(void)
{
    while (TRUE) {
        down(&mutex);
        rc = rc + 1;
        if (rc == 1) down(&db);
        up(&mutex);
        read_data_base();
        down(&mutex);
        rc = rc – 1;
        if (rc == 0) up(&db);
        up(&mutex);
        use_data_read();
    }
}
```

Reader 3

```
void reader(void)
{
    while (TRUE) {
        down(&mutex);
        rc = rc + 1;
        if (rc == 1) down(&db);
        up(&mutex);
        read_data_base();
        down(&mutex);
        rc = rc – 1;
        if (rc == 0) up(&db);
        up(&mutex);
        use_data_read();
    }
}
```

mutex = 0
rc = 1
db = 0

db = 0 writer -> sleep

Writer

```
void writer(void)
{
    while (TRUE) {
        think_up_data();
        down(&db);
        write_data_base();
        up(&db);
    }
}
```

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

The Readers and Writers Problem

```
typedef int semaphore;
semaphore mutex = 1;
semaphore db = 1;
int rc = 0;
```

Reader 1

```
void reader(void)
{
    while (TRUE) {
        down(&mutex);
        rc = rc + 1;
        if (rc == 1) down(&db);
        up(&mutex);
        read_data_base();
        down(&mutex);
        rc = rc – 1;
        if (rc == 0) up(&db);
        up(&mutex);
        use_data_read();
    }
}
```

Reader 2

```
void reader(void)
{
    while (TRUE) {
        down(&mutex);
        rc = rc + 1;
        if (rc == 1) down(&db);
        up(&mutex);
        read_data_base();
        down(&mutex);
        rc = rc – 1;
        if (rc == 0) up(&db);
        up(&mutex);
        use_data_read();
    }
}
```

Reader 3

```
void reader(void)
{
    while (TRUE) {
        down(&mutex);
        rc = rc + 1;
        if (rc == 1) down(&db);
        up(&mutex);
        read_data_base();
        down(&mutex);
        rc = rc – 1;
        if (rc == 0) up(&db);
        up(&mutex);
        use_data_read();
    }
}
```

```
mutex = 1
rc = 1
db = 0
```

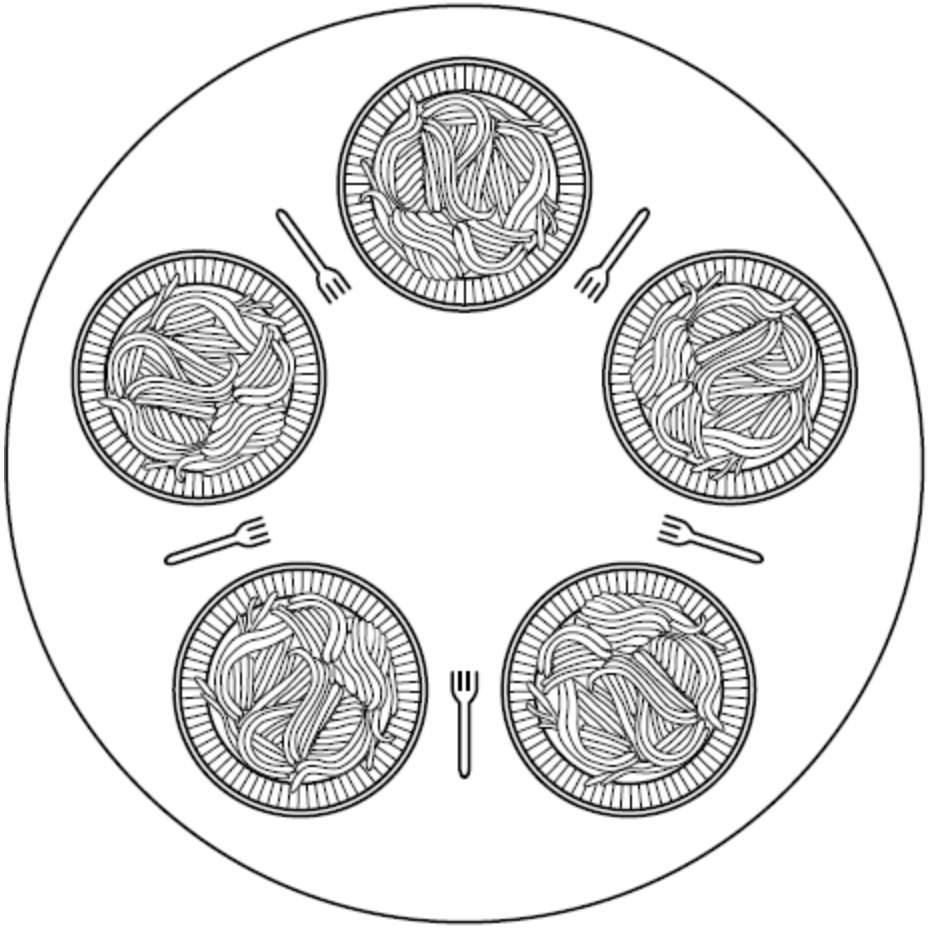
```
mutex = 1 Reader 1 or Reader 2
```

Writer

```
void writer(void)
{
    while (TRUE) {
        think_up_data();
        down(&db);
        write_data_base();
        up(&db);
    }
}
```

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

The Dining Philosophers Problem



Lunch time in the Philosophy Department.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

The Dining Philosophers Problem

Five philosophers are seated around a circular table. Each philosopher has a plate of spaghetti. The spaghetti is so slippery that a philosopher needs two forks to eat it. Between each pair of plates is one fork.

A philosopher may eat if he can pickup the two forks adjacent to him. One fork may be picked up by anyone of its adjacent followers but not both.

Can you write a program for each philosopher that does what it is supposed to do and never gets stuck?

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

The Dining Philosophers Problem

```
#define N 5                          /* number of philosophers */

void philosopher(int i)              /* i: philosopher number, from 0 to 4 */
{
    while (TRUE) {
        think( );                    /* philosopher is thinking */
        take_fork(i);                /* take left fork */
        take_fork((i+1) % N);        /* take right fork; % is modulo operator */
        eat( );                      /* yum-yum, spaghetti */
        put_fork(i);                 /* put left fork back on the table */
        put_fork((i+1) % N);         /* put right fork back on the table */
    }
}
```

A nonsolution to the dining philosophers problem.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

The Dining Philosophers Problem

```
#define N            5            /* number of philosophers */
#define LEFT         (i+N–1)%N    /* number of i's left neighbor */
#define RIGHT        (i+1)%N      /* number of i's right neighbor */
#define THINKING     0            /* philosopher is thinking */
#define HUNGRY       1            /* philosopher is trying to get forks */
#define EATING       2            /* philosopher is eating */
typedef int semaphore;            /* semaphores are a special kind of int */
int state[N];                     /* array to keep track of everyone's state */
semaphore mutex = 1;              /* mutual exclusion for critical regions */
semaphore s[N];                   /* one semaphore per philosopher */

void philosopher(int i)           /* i: philosopher number, from 0 to N–1 */
{
    while (TRUE) {                /* repeat forever */
        think( );                 /* philosopher is thinking */
        take_forks(i);            /* acquire two forks or block */
        eat( );                   /* yum-yum, spaghetti */
        put_forks(i);             /* put both forks back on table */
    }
}
```

A solution to the dining philosophers problem.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

The Dining Philosophers Problem

```
            put_forks(i);                    /* put both forks back on table */
        }
}

void take_forks(int i)                       /* i: philosopher number, from 0 to N–1 */
{
        down(&mutex);                        /* enter critical region */
        state[i] = HUNGRY;                   /* record fact that philosopher i is hungry */
        test(i);                             /* try to acquire 2 forks */
        up(&mutex);                          /* exit critical region */
        down(&s[i]);                         /* block if forks were not acquired */
}

void put_forks(i)                            /* i: philosopher number, from 0 to N–1 */
```

A solution to the dining philosophers problem.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

The Dining Philosophers Problem

```
}

void put_forks(i)                    /* i: philosopher number, from 0 to N–1 */
{
        down(&mutex);                /* enter critical region */
        state[i] = THINKING;         /* philosopher has finished eating */
        test(LEFT);                  /* see if left neighbor can now eat */
        test(RIGHT);                 /* see if right neighbor can now eat */
        up(&mutex);                  /* exit critical region */
}

void test(i) /* i: philosopher number, from 0 to N–1 */
{
        if (state[i] == HUNGRY && state[LEFT] != EATING && state[RIGHT] != EATING) {
                state[i] = EATING;
                up(&s[i]);
        }
}
```

A solution to the dining philosophers problem.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.